CHARUSAT
CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

Faculty of Engineering and Technology

Devang Patel Institute of Advance Technology & Research

Department Of Computer Science & Engineering

A.Y. 2026-2027[Odd]

Lab Manual

CEUE203: Object Oriented Programming.

Name: Vaidehi Parmar

ID: 25DCS073

PRACTICAL 3

PART A:

1. Distinct points (Point.java + Driver.java with main)

(a) Create class Point with private int x, y and a constructor.
(b) Override toString() to return "(x, y)".
(c) Override equals(Object) to be true when both x and y match, and hashCode() to agree
(e.g. Objects.hash(x, y)).
(d) In Driver.main: build a Point[] with some repeated coordinates.
(e) Count distinct points: for each point, use equals() to check whether it already appeared earlier; count those that did not.
(f) Print "Distinct: N".
Expected output: 5 points with 2 repeats → "Distinct: 3".

SOLUTION:

Point.java

```java
import java.util.Objects;
class Point {
    private int x;
    private int y;
    public Point(int x, int y) {
        this.x = x;
        this.y = y;
    }

    @Override
    public String toString() {
        return "(" + x + ", " + y + ")";
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj)
            return true;

        if (obj == null || getClass() != obj.getClass())
            return false;

        Point p = (Point) obj;
        return x == p.x && y == p.y;
    }

    @Override
    public int hashCode() {
        return Objects.hash(x, y);
    }
}
```

Driver.java

```java
class Driver {
    Run | Debug
    public static void main(String[] args) {

        Point[] points = {
            new Point(x: 1, y: 2),
            new Point(x: 3, y: 4),
            new Point(x: 1, y: 2),
            new Point(x: 5, y: 6),
            new Point(x: 3, y: 4)
        };

        int distinctCount = 0;

        for (int i = 0; i < points.length; i++) {
            boolean alreadyExists = false;

            for (int j = 0; j < i; j++) {
                if (points[i].equals(points[j])) {
                    alreadyExists = true;
                    break;
                }
            }

            if (!alreadyExists) {
                distinctCount++;
            }
        }

        System.out.println(x: "5 points with 2 repeats");
        System.out.println("Distinct: " + distinctCount);
    }
}
```

Output:

```
5 points with 2 repeats
Distinct: 3
PS C:\Users\parma\OneDrive\Documents\OOPs-sem3>
```

2. Duplicate card check (Card.java + Driver.java)

(a) Create class Card with private String rank, suit and a constructor.
(b) Override toString() to return rank + " of " + suit (e.g. "Queen of Hearts").
(c) Override equals()/hashCode() by rank + suit.
(d) In Driver.main: add cards to a Card[] one at a time; before adding each, use equals() to check it against the earlier cards.
(e) Print "Duplicate found: <card>" the first time a repeat appears.
Expected output: a second "Ace of Spades" → "Duplicate found: Ace of Spades".

Solution:

Card.java

```java
import java.util.Objects;
public class Card {
    private String rank;
    private String suit;
    public Card(String rank, String suit) {
        this.rank = rank;
        this.suit = suit;
    }
    @Override
    public String toString() {
        return rank + " of " + suit;
    }
    @Override
    public boolean equals(Object obj) {
        if (this == obj)
            return true;
        if (obj == null || getClass() != obj.getClass())
            return false;
        Card other = (Card) obj;
        return rank.equals(other.rank) && suit.equals(other.suit);
    }
    @Override
    public int hashCode() {
        return Objects.hash(rank, suit);
    }
}
```

Driver2.java

```java
public class Driver2 {
    Run | Debug
    public static void main(String[] args) {
        Card[] incoming = {
            new Card(rank: "Ace", suit: "Spades"),
            new Card(rank: "King", suit: "Hearts"),
            new Card(rank: "Queen", suit: "Diamonds"),
            new Card(rank: "Ace", suit: "Spades")
        };
        Card[] cards = new Card[incoming.length];
        int size = 0;
        for (int i = 0; i < incoming.length; i++) {
            boolean duplicate = false;
            for (int j = 0; j < size; j++) {
                if (incoming[i].equals(cards[j])) {
                    System.out.println("Duplicate found: " + incoming[i]);
                    duplicate = true;
                    break;
                }
            }
            if (duplicate) {
                break;
            }
            cards[size] = incoming[i];
            size++;
        }
    }
}
```

Output:

```
PS C:\Users\parma\OneDrive\Documents\OOPs-sem3> javac Driver2.java
PS C:\Users\parma\OneDrive\Documents\OOPs-sem3> java Driver2
Duplicate found: Ace of Spades
PS C:\Users\parma\OneDrive\Documents\OOPs-sem3>
```

3. Fractions (Fraction.java + Driver.java)

(a) Create class Fraction with private int num, den.
(b) In the constructor, reduce to lowest terms: compute g = gcd(num, den) and divide both by g.
(c) Override toString() to return num + "/" + den.
(d) Override equals()/hashCode() using the reduced num and den.
(e) In Driver.main: create several equivalent fractions (1/2, 2/4, 3/6) and confirm they are all equal and print the same.
Expected output: new Fraction(2,4) prints "1/2" and equals new Fraction(1,2).

Solution:

Fraction.java

```java
import java.util.Objects;
public class Fraction {
    private int num;
    private int den;
    public Fraction(int num, int den) {
        int g = gcd(Math.abs(num), Math.abs(den));
        this.num = num / g;
        this.den = den / g;
    }
    private int gcd(int a, int b) {
        while (b != 0) {
            int temp = b;
            b = a % b;
            a = temp;
        }
        return a;
    }
    @Override
    public String toString() {
        return num + "/" + den;
    }
    @Override
    public boolean equals(Object obj) {
        if (this == obj)
            return true;
        if (obj == null || getClass() != obj.getClass())
            return false;
        Fraction other = (Fraction) obj;
        return num == other.num && den == other.den;
    }
    @Override
    public int hashCode() {
        return Objects.hash(num, den);
    }
}
```

Driver3.java

```java
public class Driver3 {
    Run | Debug
    public static void main(String[] args) {

        Fraction f1 = new Fraction(1, 2);
        Fraction f2 = new Fraction(2, 4);
        Fraction f3 = new Fraction(3, 6);

        System.out.println(f1);
        System.out.println(f2);
        System.out.println(f3);

        System.out.println(f2.equals(f1));
        System.out.println(f3.equals(f1));
        System.out.println(f2.equals(f3));
    }
}
```

Output:

```
PS C:\Users\parma\OneDrive\Documents\OOPs-sem3> javac Driver3.java
PS C:\Users\parma\OneDrive\Documents\OOPs-sem3> java Driver3
1/2
1/2
1/2
true
true
true
PS C:\Users\parma\OneDrive\Documents\OOPs-sem3>
```

PART B:

In Practical 2 you created the Account and Customer classes. Here you make their objects behave correctly — they print as readable text (toString) and are compared by account number (equals/hashCode) — and you add a nested Address class. Correct equals/hashCode is essential because Practical 12 will store accounts in collections that rely on it.

1. In Account, override the toString() method to return a readable line containing accountNumber, ownerName and balance.

2. In Account, override equals(Object o) and hashCode() so that two accounts are equal when their accountNumber values are equal.

3. In Customer, add a public static nested class named Address with String fields line, city and pincode plus getters; add an Address field to Customer and a getAddress() method.

4. In Customer, add a clone() method (implement Cloneable) that returns a copy of the customer.

5.In main, print accounts using toString(), compare two Account objects with equals(), and use instanceof to check an object's type.

Solution:

```
import java.util.Objects;
import java.util.Scanner;
class Customer implements Cloneable {
    private String name;
    private String email;
    private String mobile;
    private final String customerId;
    private Address address;
    private static long customerCounter = 101;
    public static class Address {
        private String line;
        private String city;
        private String pincode;
        public Address(String line, String city, String pincode) {
            this.line = line;
            this.city = city;
            this.pincode = pincode;
        }
        public String getLine() {
            return line;
        }
        public String getCity() {
            return city;
        }
        public String getPincode() {
            return pincode;
        }
    }
    private static String generateCustomerId() {
        return "CUST" + customerCounter++;
```

```
    }
    public Customer(String name, String email, String mobile, Address address) {
        this.name = name;
        this.email = email;
        this.mobile = mobile;
        this.address = address;
        this.customerId = generateCustomerId();
    }
    public String getName() {
        return name;
    }
    public String getEmail() {
        return email;
    }
    public String getMobile() {
        return mobile;
    }
    public String getCustomerId() {
        return customerId;
    }
    public Address getAddress() {
        return address;
    }
    @Override
    public Customer clone() {
        try {
            return (Customer) super.clone();
        } catch (CloneNotSupportedException e) {
            return null;
        }
    }
}
class Account {
```

```
    private final String accountNumber;
    private String ownerName;
    private long balance;
    private boolean active;
    private static int accountCounter = 1;
    private static String generateAccountNumber() {
        return String.format("AC%04d", accountCounter++);
    }
    public Account(String ownerName, long balance) {
        this.accountNumber = generateAccountNumber();
        this.ownerName = ownerName;
        this.balance = balance;
        this.active = true;
    }
    public Account(String ownerName) {
        this(ownerName, 0);
    }
    public void deposit(long amount) {
        if (amount > 0) {
            balance += amount;
        }
    }
    public boolean withdraw(long amount) {
        if (amount > 0 && balance >= amount) {
            balance -= amount;
            return true;
        }
        return false;
    }
    public String getAccountNumber() {
        return accountNumber;
    }
    public String getOwnerName() {
```

```
        return ownerName;
    }
    public long getBalance() {
        return balance;
    }
    public boolean isActive() {
        return active;
    }
    @Override
    public String toString() {
        return accountNumber + " | " + ownerName + " | Balance = ₹" + balance;
    }
    @Override
    public boolean equals(Object obj) {
        if (this == obj)
            return true;
        if (obj == null || getClass() != obj.getClass())
            return false;
        Account a = (Account) obj;
        return accountNumber.equals(a.accountNumber);
    }
    @Override
    public int hashCode() {
        return Objects.hash(accountNumber);
    }
}
public class MiniBank {
    record BankInfo(String name, String branch) {
    }
    enum MenuOption {
        OPEN_ACCOUNT,
        DEPOSIT,
        WITHDRAW,
```

```
        TRANSFER,
        EXIT
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        BankInfo bank = new BankInfo("MiniBank", "Main Branch");
        Customer.Address address =
                new Customer.Address("12 MG Road", "Ahmedabad", "380001");
        Customer c1 = new Customer(
                "Vaidehi",
                "vaidehi@gmail.com",
                "1234567890",
                address);
        Customer copy = c1.clone();
        Account[] accounts = {
                new Account(c1.getName(), 5000),
                new Account("Riya"),
                new Account("Rahul", 10000)
        };
        System.out.println("Customer Details");
        System.out.println("ID: " + c1.getCustomerId());
        System.out.println("Name: " + c1.getName());
        System.out.println("Email: " + c1.getEmail());
        System.out.println("Mobile: " + c1.getMobile());
        System.out.println("City: " + c1.getAddress().getCity());
        System.out.println("\nTransactions");
        accounts[0].deposit(1000);
        accounts[1].deposit(3000);
        System.out.println("Withdraw ₹2000 from Account 1: "
                + accounts[0].withdraw(2000));
        System.out.println("Withdraw ₹4000 from Account 3: "
                + accounts[2].withdraw(4000));
        System.out.println("\nAccount Details");
```

```
for (Account acc : accounts) {
    System.out.println(acc);
}
System.out.println("\nEquals Demo");
System.out.println(accounts[0].equals(accounts[1]));
System.out.println(accounts[0].equals(accounts[0]));
System.out.println("\nInstanceof Demo");
Object obj = accounts[0];
if (obj instanceof Account) {
    System.out.println("Object is an Account.");
}
System.out.println("\nClone Demo");
System.out.println("Original Customer : " + c1.getName());
System.out.println("Cloned Customer   : " + copy.getName());
System.out.println("\n================================");
System.out.println(bank.name() + " - " + bank.branch());
System.out.println("================================");
while (true) {
    System.out.println("\nMenu");
    System.out.println("1. Open Account");
    System.out.println("2. Deposit");
    System.out.println("3. Withdraw");
    System.out.println("4. Transfer");
    System.out.println("5. Exit");
    System.out.print("Enter your choice: ");
    int choice = sc.nextInt()
    MenuOption option = switch (choice) {
        case 1 -> MenuOption.OPEN_ACCOUNT;
        case 2 -> MenuOption.DEPOSIT;
        case 3 -> MenuOption.WITHDRAW;
        case 4 -> MenuOption.TRANSFER;
        case 5 -> MenuOption.EXIT;
        default -> null;
```

```
        };
        if (option == null) {
            System.out.println("Invalid choice!");
            continue;
        }
        switch (option) {
            case OPEN_ACCOUNT ->
                System.out.println("Open Account - To be implemented in a later lab.");
            case DEPOSIT ->
                System.out.println("Deposit - To be implemented in a later lab.");
            case WITHDRAW ->
                System.out.println("Withdraw - To be implemented in a later lab.");
            case TRANSFER ->
                System.out.println("Transfer - To be implemented in a later lab.");
            case EXIT -> {
                System.out.println("Thank you for using MiniBank!");
                sc.close();
                return;
            }
        }
    }
  }
}
```

Output:

```
Customer Details
ID: CUST101
Name: Vaidehi
Email: vaidehi@gmail.com
Mobile: 1234567890
City: Ahmedabad

Transactions
Withdraw ?2000 from Account 1: true
Withdraw ?4000 from Account 3: true

Account Details
AC0001 | Vaidehi | Balance = ?4000
AC0002 | Riya | Balance = ?3000
AC0003 | Rahul | Balance = ?6000

Equals Demo
false
true

Instanceof Demo
Object is an Account.

Clone Demo
Original Customer : Vaidehi
Cloned Customer   : Vaidehi

================================
MiniBank - Main Branch
================================

Menu
1. Open Account
2. Deposit
3. Withdraw
4. Transfer
5. Exit
Enter your choice: 3
Withdraw - To be implemented in a later lab.
```